2.2 Le modèle relationnel avancé

Programme

Pédagogie

Compétences attendues

- Interpréter un schéma relationnel
- Adapter un schéma relationnel à un besoin d'évolution d'une base de données

Savoirs associés

- Collecte et sélection de données
- Dépendances fonctionnelles
- Modèle relationnel

Plan

- **Cours** 2.2.1 Prendre en compte le temps dans un modèle relationnel 2.2.2 Gérer les attributs non renseignés 2.2.3 Déterminer la clé primaire dans des situations particulières
- **Synthèse**

Mots clés

Clé étrangère • Clé primaire • Concaténation • Héritage • Historisation • Identification relative • Table complémentaire • Table générique • Temps

Le chapitre précédent a posé les règles générales. Celui-ci traite les trois situations où elles ne suffisent plus : quand il faut **dater** un fait, quand une table se remplit de **cases vides**, et quand la **clé primaire** ne peut pas être un simple numéro. Ce sont, dans cet ordre, les trois pièges les plus fréquents des sujets d'examen.

À quoi ça sert ?

Les modèles du chapitre 2.1 décrivent un état : voici les clients, voici les produits. Or la gestion ne travaille jamais sur un état — elle travaille sur un **historique**. Combien de fois ce client a-t-il été visité ? Quel était le tarif appliqué en mars ? Depuis quand ce salarié occupe-t-il ce poste ?

Ce chapitre vous apprend à modéliser le temps, et à ne pas construire une base qui écrase son propre passé. C'est le défaut le plus coûteux qu'on rencontre en entreprise : une base qui ne sait dire que « aujourd'hui », et qui ne peut donc justifier aucun chiffre d'hier.

Comment lire les modèles de ce cours

Dans les manuels, la clé primaire est **soulignée**. Le soulignement se transmet mal à l'écran et n'est pas restitué par les lecteurs d'écran : ce cours utilise donc une notation explicite, et vous devez pouvoir passer de l'une à l'autre.

```
FACTURE (NumFact, DateFact, #NumCli)
```

- la **clé primaire** est le ou les attributs de tête ; quand elle est composée, ou quand elle inclut des clés étrangères, elle est **rappelée en toutes lettres sous le modèle** ;
- un attribut précédé d'un `#` est une **clé étrangère** ;
- un attribut peut être **à la fois** clé étrangère et membre de la clé primaire : c'est même le cas le plus fréquent dans ce chapitre ;
- dans les **tables d'exemple**, l'en-tête des colonnes qui forment la clé primaire est **en gras**.

Sur votre copie d'examen, soulignez. C'est la notation attendue par le correcteur.

Cours

2.2.1 Prendre en compte le temps dans un modèle relationnel

Chez Vasseur — la situation

Menuiseries Vasseur veut savoir combien de fois ses commerciaux ont rendu visite au promoteur Rivière & Fils avant de décrocher le marché. La base actuelle ne contient qu'une colonne « dernière visite », écrasée à chaque passage.

La réponse est donc perdue — non pas mal rangée : **perdue**. Personne ne l'a jamais enregistrée.

La question de cette section : comment un modèle relationnel garde-t-il la trace de ce qui s'est passé, au lieu de ne connaître que le dernier état ?

- Modéliser le temps, c'est répondre à une seule question : **de quoi la date dépend-elle ?** Trois cas se présentent, et chacun conduit à une structure différente.

Cas 1 — la date est propre à un objet

- Pour un objet précis, une seule date est possible ;
- Il en résulte une dépendance fonctionnelle entre la clé primaire de la relation et l'attribut date.

Exemple — la commande n° 1253. La commande n° 1253 a été passée le 12 janvier 2019. La date de la commande ne se rapporte qu'à cette seule commande.

```
COMMANDE (NumCommande, DateCommande, #NumClient)
```

La date est un **attribut ordinaire** : `NumCommande —▶ DateCommande`. Il n'y a rien de particulier à faire.

Cas 2 — la date concerne plusieurs objets, et l'événement ne survient qu'une fois

- La date résulte de la mise en relation d'au moins deux objets différents ;
- Cette mise en relation ne peut survenir **qu'une seule fois** — ou, si elle survient plusieurs fois, l'historique n'est pas conservé.

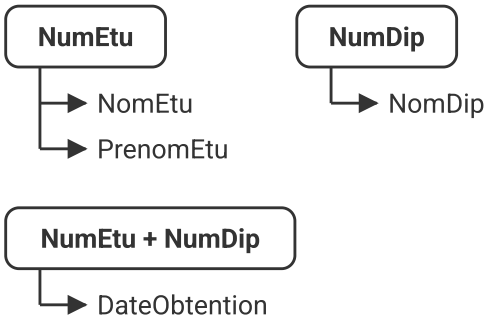
Exemple — les diplômes de l'école DSCG+. Les bases de données de l'école DSCG+ gèrent les étudiants (numéro, nom, prénom) et les diplômes (code, nom du diplôme). Pour chaque étudiant, on veut connaître la date d'obtention de ses diplômes.

Un étudiant peut obtenir plusieurs diplômes, un diplôme peut être obtenu par plusieurs étudiants — mais **un étudiant ne passe le même diplôme qu'une fois**. La date d'obtention ne dépend donc que du couple (étudiant, diplôme) :

```
ETUDIANT       (NumEtu, NomEtu, PrenomEtu)
DIPLOME        (NumDip, NomDip)
OBTENIRDIPLOME (#NumEtu, #NumDip, DateObtention)
```

Clé primaire de `OBTENIRDIPLOME` : le couple (`#NumEtu`, `#NumDip`). La date reste un attribut ordinaire.

Graphe des dépendances fonctionnelles de l'obtention des diplômes



Description du schéma, en toutes lettres

```
NumEtu ⟶ NomEtu, PrenomEtu
NumDip ⟶ NomDip
(NumEtu, NumDip) ⟶ DateObtention
```

Deux sources simples, `NumEtu` et `NumDip`, et une source composée : la date d'obtention ne dépend ni de l'étudiant seul (il peut avoir plusieurs diplômes) ni du diplôme seul (plusieurs étudiants l'obtiennent), mais du couple.

Une représentation sous forme de table permet de vérifier que les clés primaires jouent bien leur rôle : aucun couple (étudiant, diplôme) n'apparaît deux fois.

ETUDIANT

NumEtu	NomEtu	PrenomEtu
E1	GASPROT	Martin
E2	VALRU	Quentin
E3	BUREEZSA	Sonia
E4	POURAE	Valentine

DIPLOME

NumDip	NomDip
D1	DCG
D2	DCG-INTEC
D3	Licence gestion

OBTENIRDIPLOME

#NumEtu	#NumDip	DateObtention
E1	D1	29/08/N
E2	D2	10/10/N
E3	D2	09/12/N-1
E4	D1	24/08/N-1
E4	D3	30/06/N

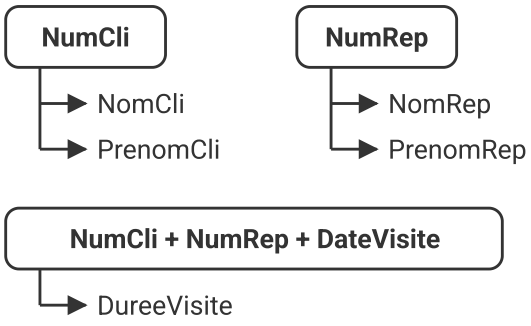
E4 figure deux fois dans `OBTENIRDIPLOME`, mais avec deux diplômes différents. Un étudiant n'obtenant le même diplôme qu'une fois, aucun couple (`#NumEtu`, `#NumDip`) ne peut s'y répéter.

Cas 3 — la date concerne plusieurs objets, et l'événement peut survenir plusieurs fois

- La date résulte de la mise en relation d'au moins deux objets différents ;
- Cette mise en relation peut survenir **plusieurs fois** ;
- Si l'on veut conserver l'historique, il faut distinguer les différentes réalisations du même couple d'objets ;
- **La date doit alors être intégrée à la clé primaire.**

Exemple — les visites des représentants. Un représentant visite les clients d'une entreprise. Il peut visiter plusieurs fois le même client, mais toujours à des dates différentes. Un client n'est pas forcément toujours visité par le même représentant. La base conserve les coordonnées des clients et des représentants, ainsi que l'**historique des visites** et leur durée.

Graphe des dépendances fonctionnelles des visites



Description du schéma, en toutes lettres

```
NumCli ⟶ NomCli, PrenomCli
NumRep ⟶ NomRep, PrenomRep
(NumCli, NumRep, DateVisite) ⟶ DureeVisite
```

La durée d'une visite ne dépend pas du seul couple (client, représentant) : le même représentant peut revenir chez le même client. Il faut la date pour désigner une visite précise.

```
CLIENT        (NumCli, NomCli, PrenomCli)
REPRESENTANT  (NumRep, NomRep, PrenomRep)
VISITERCLIENT (#NumCli, #NumRep, DateVisite, DureeVisite)
```

Clé primaire de `VISITERCLIENT` : le triplet (`#NumCli`, `#NumRep`, `DateVisite`). C'est la date qui, ajoutée au couple, rend chaque visite unique.

CLIENT

NumCli	NomCli	PrenomCli
C1	DUJARDIN	Bertrand
C2	CANET	Chantal
C3	TAUTOU	Philippe
C4	ADJANI	Catherine

REPRESENTANT

NumRep	NomRep	PrenomRep
R1	SAND	Aurore
R2	CAMUS	Michel
R3	DURAS	Valentine

VISITERCLIENT

`#NumCli`	`#NumRep`	DateVisite	DureeVisite
C1	R1	25/09/N-1	1
C1	R1	02/01/N	1,5
C2	R2	10/09/N-1	0,5
C2	R2	10/12/N-1	1
C3	R2	09/12/N-1	0,5

#NumCli	#NumRep	DateVisite	DureeVisite
C4	R1	25/09/N-1	1
C4	R3	12/12/N-1	0,5
C4	R3	15/03/N	1
C4	R3	30/06/N	0,5

Le couple (C1, R1) apparaît deux fois, le couple (C4, R3) trois fois : sans la date, la clé primaire aurait des doublons. C'est `DateVisite`, ajoutée au couple, qui rend chaque ligne unique.

> ⚠ **Le test à faire systématiquement**
>
> Posez-vous la question : **« cela peut-il se produire deux fois ? »**
>
> - Non → la date est un attribut ordinaire (cas 1 ou 2) ;
> - Oui, et je veux garder la trace → la date **entre dans la clé primaire** (cas 3) ;
> - Oui, mais je ne garde que le dernier → la date reste un attribut, et **vous acceptez explicitement de perdre l'historique**. Ce n'est pas une erreur si c'est un choix — c'en est une si c'est un oubli.

> **Une image pour comprendre**
>
> Un attribut date, c'est la mention « dernière révision » écrite au crayon sur le carnet d'entretien d'une voiture : on l'efface et on la réécrit à chaque passage. Une date dans la clé primaire, c'est le carnet lui-même, avec une ligne par intervention. Le premier tient sur un post-it ; seul le second permet de revendre la voiture.

> **Chez Vasseur — la réponse**
>
> **Comment un modèle relationnel garde-t-il la trace de ce qui s'est passé, au lieu de ne connaître que le dernier état ?**
>
> En faisant entrer le **temps dans la clé primaire**. Tant que la date est un simple attribut, chaque nouvel événement écrase le précédent : il n'y a de place que pour une valeur. Dès que la date fait partie de la clé, chaque occurrence devient une **ligne distincte**, et l'historique existe.
>
> Le choix se fait en une question — *cela peut-il se produire deux fois ?* — et se décline en trois cas :

- la date ne dépend que d'un objet → attribut ordinaire (`COMMANDE`) ;
- elle dépend d'un couple, non répétable → attribut de la table de liaison (`OBTENIRDIPLOME`) ;
- elle dépend d'un couple, répétable → **elle rejoint la clé primaire** (`VISITERCLIENT`).

Chez Vasseur, la colonne « dernière visite » — le crayon du carnet d'entretien — relevait du troisième cas traité comme le premier : un commercial visite plusieurs fois le même prospect, et c'est précisément cette répétition qui a de la valeur commerciale. Le modèle correct est donc :

```
VISITER (#NumCommercial, #NumClient, DateVisite, DureeVisite, CompteRendu)
```

Clé primaire : le triplet (`#NumCommercial`, `#NumClient`, `DateVisite`).

L'erreur n'était pas une maladresse d'écriture : c'était une **perte d'information définitive**, qu'aucune requête, aucun tableau croisé, aucun consultant ne récupérera. Sans historique, l'entreprise ne peut ni évaluer le nombre de visites nécessaires pour signer, ni comparer ses commerciaux, ni justifier un budget de déplacement.

✓ **Vérifiez que vous avez compris**

Une salle de sport enregistre les passages de ses adhérents. Faut-il mettre la date dans la clé primaire de la table `PASSAGE` ?

Oui. Un même adhérent passe plusieurs fois dans la même salle, et c'est justement ce qu'on veut compter. La clé primaire est le couple (`#NumAdherent`, `DateHeurePassage`).

Attention au piège : la **date seule** ne suffit pas si un adhérent peut venir deux fois dans la journée. Il faut la date **et l'heure** — ou, plus sûr, un numéro de passage séquentiel. Retenez-en le principe : la clé primaire doit garantir l'unicité pour *tous* les cas possibles, pas seulement pour les cas habituels.

Mise en pratique

Cas pratique 1 : Les heures des poseurs

Compétences attendues

- Interpréter un schéma relationnel
- Adapter un schéma relationnel à un besoin d'évolution d'une base de données

Pour suivre le temps passé sur les chantiers, Thomas Lemoine a créé les tables suivantes :

```
SALARIE    (NumSalarie, NomSalarie, Fonction, CoutHoraire)
CHANTIER   (NumChantier, VilleChantier, #NumClient)
INTERVENIR (#NumChantier, #NumSalarie, NbHeures)
```

La clé primaire de `INTERVENIR` est le couple (`#NumChantier`, `#NumSalarie`).

Missions

1. **Que permet cette clé primaire, et qu'interdit-elle ? Illustrez avec Yann Mercier, qui a travaillé sur le chantier 105 le 17 juin puis le 18 juin 2025.**
2. **Le 18 juin, Yann Mercier pose les fenêtres le matin, puis revient l'après-midi régler une porte-fenêtre, ce que l'entreprise appelle une intervention de SAV. Camille Vasseur veut distinguer les heures de pose des heures de SAV. Le modèle le supporte-t-il ? Corrigez-le.**
3. **Camille veut valoriser ces heures. Le coût horaire des poseurs est passé de 38 € à 41 € au 1er janvier 2026. En l'état, que donnerait le calcul pour une intervention de 2025 ? Proposez une correction.**
4. **Le modèle corrigé permet-il de connaître, pour chaque chantier, le poseur qui y a passé le plus d'heures ? Expliquez comment.**

2.2.2 Gérer les attributs non renseignés

Chez Vasseur — la situation

La table des clients de Menuiseries Vasseur porte 22 colonnes. Pour un particulier, les colonnes `SIRET`, `NumMarchePublic`, `ServiceFacturation` et `DelaiPaiementNegocie` restent vides. Pour une collectivité, ce sont `DateNaissance` et `TypeLogement` qui le sont.

Résultat : plus de la moitié des cases du fichier sont vides, et personne ne sait plus si une case vide signifie « sans objet » ou « on ne l'a pas saisi ».

La question de cette section : que faire quand une table rassemble des objets qui ne se ressemblent qu'à moitié ?

Le principe de l'héritage

- Une table possède de nombreux attributs non renseignés lorsqu'elle est constituée d'objets qui ne sont **pas parfaitement homogènes** — par exemple des catégories de clients aux caractéristiques différentes ;
- À partir d'une **table générique** contenant les données communes à tous les objets, il faut créer des **tables complémentaires** conservant les données propres à chaque catégorie ;
- Les tables complémentaires « héritent » de la clé primaire de la table générique.

Définition

La notion d'**héritage** consiste à attacher systématiquement aux données communes d'une table générique des données spécifiques contenues dans des tables complémentaires, réduisant ainsi la présence d'attributs non renseignés.

Une image pour comprendre

C'est le dossier administratif d'une entreprise. Tout le monde a une chemise « identité » : nom, matricule, coordonnées. Seuls les salariés ont en plus la chemise « contrat et salaire » ; seuls les stagiaires ont la chemise « convention et école ».

Personne ne se promène avec un formulaire unique de 40 lignes dont 25 barrées. Une base de données bien conçue ne le fait pas non plus.

Exemple — le personnel d'une entreprise.

Matricule	NomPers	PrenPers	NomDiplome	Année	Salaire	DateEmb
Mat1	Garden	Franck			60 000	25/09/N-10
Mat2	Duck	Yves			42 000	12/10/N-3

Matricule	NomPers	PrenPers	NomDiplome	Année	Salaire	DateEmb
Mat3	Tot	Léa			28 000	15/02/N-10
Mat4	Gilles	Eve			32 000	23/07/N-2
Mat5	Gain	Fred	DCG	1		
Mat6	Oelo	Reine	DCG	2		

Le personnel travaillant dans une entreprise est soit salarié, soit étudiant. Pour tous, on enregistre le matricule, le nom et le prénom. Pour les salariés, on conserve en plus le montant du salaire et la date d'embauche. Pour les stagiaires, le nom du diplôme préparé et le nombre d'années de formation.

Une table `PERSONNEL` unique devrait comporter `Matricule, Nom, Prénom, Salaire, DateEmbauche, NomDiplôme, Année` — et resterait, comme ci-dessus, à moitié vide.

La solution est une table générique et deux tables complémentaires :

```
PERSONNEL (Matricule, NomPers, PrenomPers)
ETUDIANT  (#MatriculeETU, NomDiplome, AnneeFormation)
SALARIE   (#MatriculeSAL, Salaire, DateEmbauche)
```

Clés primaires : `Matricule` pour `PERSONNEL` ; `#MatriculeETU` pour `ETUDIANT` ; `#MatriculeSAL` pour `SALARIE`. Dans les tables complémentaires, la clé primaire **est** la clé étrangère : c'est la signature de l'héritage.

Renommer la clé dans les tables filles (`MatriculeETU` = matricule de l'étudiant, `MatriculeSAL` = matricule du salarié) rend le modèle plus lisible ; ce n'est pas obligatoire.

PERSONNEL

Matricule	NomPers	PrenomPers
Mat1	Garden	Franck
Mat2	Duck	Yves
Mat3	Tot	Léa
Mat4	Gilles	Eve
Mat5	Gain	Fred
Mat6	Oelo	Reine

ETUDIANT

#MatriculeETU	NomDiplome	AnneeFormation
Mat5	DCG	1
Mat6	DCG	2

SALARIE

#MatriculeSAL	Salaire	DateEmbauche
Mat1	60 000	25/09/N-10
Mat2	42 000	12/10/N-3
Mat3	28 000	15/02/N-10
Mat4	32 000	23/07/N-2

Plus aucune case vide : chacun figure dans `PERSONNEL`, puis dans `ETUDIANT` **ou** dans `SALARIE`, selon sa catégorie.

⚠ **Attention à ne pas confondre**

Héritage et **association 1:1** se ressemblent, et le correcteur attend que vous les distinguiez.

Dans l'héritage, la table fille **ne peut pas exister sans** la table mère, et sa clé primaire *est* celle de la mère : un étudiant est un membre du personnel. Dans une association 1:1 ordinaire — un salarié et son véhicule de fonction —, les deux objets existent indépendamment.

Le test : *« l'objet de la table fille est-il un cas particulier de celui de la table mère ? »* Si oui, héritage.

☰ **Chez Vasseur — la réponse**

Que faire quand une table rassemble des objets qui ne se ressemblent qu'à moitié ?

On applique l'**héritage** : une table générique pour ce que tous les clients ont en commun, et une table complémentaire par catégorie.

```
CLIENT        (NumClient, NomClient, AdrClient, CPClient, VilleClient, TelClient
PARTICULIER   (#NumClientPART, DateNaissance, TypeLogement)
PROFESSIONNEL (#NumClientPRO, SIRET, ServiceFacturation, DelaiPaiementNegocie)
COLLECTIVITE  (#NumClientCOL, NumMarchePublic, ServiceFacturation)
```

Les 22 colonnes à moitié vides deviennent quatre tables pleines. Trois bénéfices concrets :

- **plus de cases vides**, donc plus d'ambiguïté entre « sans objet » et « non saisi » ;
- les **contrôles deviennent possibles** : le SIRET peut être rendu obligatoire pour un professionnel, ce qui était impossible tant que la colonne servait aussi aux particuliers ;
- le modèle **absorbe l'évolution** : le jour où les collectivités exigeront un identifiant supplémentaire, une seule table bouge.

Et la nuance à savoir défendre : un client qui serait à la fois particulier et professionnel n'est pas interdit par ce modèle — il aurait simplement une ligne dans chacune des deux tables filles.

✓ **Vérifiez que vous avez compris**

Une case vide dans une base de données, est-ce grave ?

Pas en soi — c'est son **ambiguïté** qui l'est. Une case vide peut signifier trois choses très différentes : la valeur n'existe pas (un particulier n'a pas de SIRET), elle existe mais n'a pas été saisie, ou elle est inconnue.

Aucune requête ne sait faire la différence, et un calcul de moyenne les traitera toutes de la même façon. L'héritage supprime le premier cas — le plus fréquent — en n'ouvrant la colonne qu'aux objets qu'elle concerne.

Mise en pratique

Cas pratique 2 : Le catalogue

Compétences attendues

- Interpréter un schéma relationnel
- Adapter un schéma relationnel à un besoin d'évolution d'une base de données

Le catalogue de Menuiseries Vasseur tient dans une table unique :

```
PRODUIT (RefProduit, Designation, Matiere, PrixHT,
         Largeur, Hauteur, NbVantaux, TypeVitrage,
         SensOuverture, NbPointsFermeture,
         SurfaceM2, TypeToiture)
```

Le catalogue comprend des **fenêtres** (largeur, hauteur, nombre de vantaux, type de vitrage), des **portes** (largeur, hauteur, sens d'ouverture, nombre de points de fermeture) et des **vérandas** (surface, type de toiture). Sur la plupart des lignes, plus de la moitié des colonnes sont vides.

Missions

1. **Nommez le défaut de ce modèle et proposez une structure corrigée**, en précisant les clés primaires.
2. `Largeur` et `Hauteur` **doivent-elles rester dans la table générique ? Discutez le cas de la véranda.**
3. **Chaque modèle existe en plusieurs coloris, numérotés à partir de 1 pour chaque modèle (coloris n° 1, 2, 3 de la F-210 ; coloris n° 1, 2 de la P-530), avec un supplément de prix propre à chacun. Complétez le modèle.** (Cette question anticipe la section suivante.)
4. **Un même produit peut être acheté auprès de plusieurs fournisseurs de profilés, et un fournisseur en fournit plusieurs. Le prix d'achat est négocié produit par produit avec chaque fournisseur. Adaptez votre modèle et justifiez la structure retenue.**

2.2.3 Déterminer la clé primaire dans des situations particulières

☰ Chez Vasseur — la situation

Sur le chantier de Saint-Avertin, l'atelier a fabriqué 14 fenêtres du modèle F-210. Elles sont numérotées de 1 à 14 — et sur le chantier suivant, la numérotation repart à 1.

Quand un poseur signale « la n° 7 est fêlée », le numéro 7 ne veut rien dire tout seul : il en existe une par chantier.

La question de cette section : que fait-on quand un numéro n'est unique qu'à l'intérieur de quelque chose d'autre ?

Distinguer les différents exemplaires d'un objet : l'identification relative

- La clé primaire d'une relation peut, dans certaines situations, être définie **par référence** à la clé primaire d'une autre relation ;

- Ainsi, lorsqu'un objet comporte plusieurs exemplaires, la clé primaire de l'exemplaire se réfère à celle de l'objet ;
- Elle ne peut exister seule : pour être unique, elle est **concaténée** avec celle de l'objet.

Exemple — les exemplaires de la bibliothèque Atout Lire. La bibliothèque Atout Lire acquiert plusieurs exemplaires d'un même livre afin de les prêter à ses adhérents. La numérotation des exemplaires **redémarre à 1 pour chaque livre**.

Un exemplaire ne correspond qu'à un seul livre ; un livre peut être acheté en plusieurs exemplaires. Du fait de la numérotation, plusieurs exemplaires portent le même numéro.

```
LIVRE           (CodeLiv, TitreLiv)
EXEMPLAIRELIVRE (NumExp, #CodeLiv, EtatExp)
```

Clé primaire de `EXEMPLAIRELIVRE` : le couple (`NumExp`, `#CodeLiv`). `NumExp` seul ne suffit pas — il existe un exemplaire n° 1 pour chaque titre du fonds.

LIVRE

CodeLiv	TitreLiv
L1	Le livre des Baltimore
L2	La disparition de Stéphanie Mailer

EXEMPLAIRELIVRE

NumExp	#CodeLiv	EtatExp
1	L1	Usagé
2	L1	État correct
1	L2	État correct
2	L2	Neuf

Le numéro 1 apparaît deux fois dans `NumExp`, jamais pour le même livre : c'est le couple (`NumExp`, `#CodeLiv`) qui est unique.

> **Une image pour comprendre**
>
> C'est l'adresse. « Appartement 3 » ne désigne rien : il faut « appartement 3 **du** 12 rue Pasteur ». Le numéro d'appartement n'est unique qu'à l'intérieur de l'immeuble, et c'est la combinaison des deux qui identifie le logement.

Une identification relative, c'est exactement cela : un numéro qui n'a de sens que rapporté à son contenant.

Dépendance fonctionnelle entre une clé primaire concaténée et une clé étrangère

- Lorsque trois objets sont en relation, l'un d'eux peut dépendre du **couple** constitué par les deux autres ;
- La clé primaire concaténée du couple est alors la **source** de la dépendance fonctionnelle pointant vers le troisième objet.

Exemple — les affectations de l'entreprise CMonChoix. Dans l'entreprise CMonChoix, des employés sont affectés à des projets et peuvent y remplir **plusieurs fonctions**. Les trois objets en interaction entraînent la création de quatre relations :

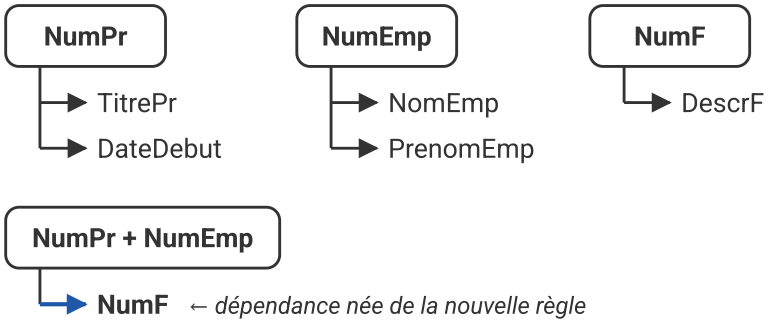
```
PROJET   (NumPr, TitrePr, DateDebut)
EMPLOYE  (NumEmp, NomEmp, PrenomEmp)
FONCTION (NumF, DescrF)
AFFECTER (#NumPr, #NumEmp, #NumF)
```

Clé primaire de `AFFECTER` : le triplet (`#NumPr`, `#NumEmp`, `#NumF`) — les trois attributs, puisqu'un employé peut tenir plusieurs fonctions sur le même projet.

À ce stade, aucun attribut ne dépend du couple (`NumPr`, `NumEmp`) : `AFFECTER` ne contient que ses trois clés étrangères, qui forment ensemble sa clé primaire.

Les règles de gestion évoluent : sur un projet donné, un employé n'a désormais qu'une seule fonction possible.

Graphe des dépendances fonctionnelles après le changement de règle



Description du schéma, en toutes lettres

```
NumPr ──▶ TitrePr, DateDebut
NumEmp ──▶ NomEmp, PrenomEmp
NumF ──▶ DescrF
(NumPr, NumEmp) ──▶ NumF      ← nouvelle dépendance
```

Les trois premières dépendances existaient déjà. La quatrième naît de la nouvelle règle : connaissant le projet et l'employé, on connaît sa fonction.

Le graphe des dépendances fonctionnelles fait apparaître une dépendance nouvelle : le couple (`#NumPr`, `#NumEmp`) suffit à déterminer `NumF`. Les attributs de la relation ne changent pas — mais **sa clé primaire, si** :

```
AFFECTER (#NumPr, #NumEmp, #NumF)
```

Clé primaire de `AFFECTER` : désormais le couple (`#NumPr`, `#NumEmp`) seul. `#NumF` devient un attribut ordinaire, qui reste une clé étrangère vers `FONCTION`.

AFFECTER

#NumPr	#NumEmp	#NumF
P1	E1	F1
P1	E2	F1
P1	E3	F2
P2	E1	F1
P2	E2	F2

Un employé intervient sur plusieurs projets et un projet réunit plusieurs employés : la clé est bien un couple. Comme l'employé n'a qu'une fonction par projet, aucun couple (`#NumPr`, `#NumEmp`) ne se répète, et c'est lui qui détermine la fonction : E2 est F1 sur le projet P1, F2 sur le projet P2.

⚠ **L'exemple à ne surtout pas survoler**

Ci-dessus, **les deux modèles s'écrivent avec les mêmes attributs** : `AFFECTER (#NumPr, #NumEmp, #NumF)`. Seule la clé primaire change — et c'est là toute la différence de sens :

Clé primaire	Ce que le modèle autorise
(`#NumPr`, `#NumEmp`, `#NumF`)	Dupont est **chef de projet et développeur** sur le projet Alpha : deux lignes
(`#NumPr`, `#NumEmp`)	Dupont n'a **qu'une fonction** sur Alpha : la seconde ligne serait refusée

C'est exactement pour cela qu'on souligne la clé primaire, et pourquoi une copie où elle ne l'est pas est incorrigible. Sur votre copie : **soulignez.**

Chez Vasseur — la réponse

Que fait-on quand un numéro n'est unique qu'à l'intérieur de quelque chose d'autre ?

On pratique une **identification relative** : la clé primaire de l'exemplaire est **concaténée** avec celle de l'objet auquel il se rattache.

```
CHANTIER   (NumChantier, AdrChantier, DateDebut, #NumClient)
MENUISERIE (NumMenuiserie, #NumChantier, #RefProduit, EtatPose)
```

Clé primaire de `MENUISERIE` : le couple (`NumMenuiserie`, `#NumChantier`).

La fenêtre « n° 7 » du poseur devient alors identifiable sans ambiguïté : c'est la n° 7 **du chantier de Saint-Avertin**. Le numéro seul ne désigne rien, le couple désigne une menuiserie et une seule — et l'atelier peut enfin tracer une pièce défectueuse jusqu'à son chantier.

Le réflexe à retenir dépasse ce cas : **dès qu'une numérotation repart à 1**, elle est relative, et sa clé primaire est composée. C'est vrai des exemplaires d'un livre, des lignes d'une facture, des lots d'une commande, des appartements d'un immeuble.

✓ Vérifiez que vous avez compris

Les lignes d'une facture sont numérotées 1, 2, 3… et la numérotation repart à 1 sur la facture suivante. Quelle est la clé primaire de la table `LIGNEFACTURE` ?

Le couple (`NumLigne`, `#NumFacture`). C'est une identification relative : `NumLigne` n'est unique qu'à l'intérieur d'une facture donnée.

Notez que ce n'est pas la seule solution possible : on pourrait aussi identifier la ligne par le couple (`#NumFacture`, `#RefProduit`) — mais ce choix interdirait de faire figurer deux fois le même produit sur une facture, ce qui arrive (deux tarifs, deux dates de livraison). Le choix d'une clé primaire est toujours une **décision de gestion**, pas seulement une décision technique.

Mise en pratique

Cas pratique 3 : L'outillage partagé

Compétences attendues

- Interpréter un schéma relationnel
- Adapter un schéma relationnel à un besoin d'évolution d'une base de données

Menuiseries Vasseur dispose d'outillage lourd sur ses deux sites : nacelles, échafaudages, monte-charges. Chaque site numérote son outillage **à partir de 1** : il existe une nacelle n° 1 à Tours et une nacelle n° 1 à Blois. Les numéros sont peints sur le matériel.

Les cinq équipes de poseurs, numérotées de 1 à 5, **réservent** un outil pour une période (date de début, date de fin). Une même équipe réserve régulièrement le même outil, à des dates différentes.

Une partie de l'outillage appartient à l'entreprise (date d'achat, valeur d'achat). L'autre partie est louée à l'année auprès d'un loueur (nom du loueur, numéro de contrat, tarif journalier).

Missions

1. **Proposez le modèle relationnel complet**, en précisant explicitement chaque clé primaire.
2. **Justifiez la clé primaire de la table de l'outillage. Quel serait le défaut d'un numéro unique pour tout l'outillage de l'entreprise ?**
3. **Une équipe peut réserver deux fois le même outil à deux périodes différentes. Votre modèle le supporte-t-il ? Quelle est la clé primaire de la réservation ?**
4. **Distinguez l'outillage possédé de l'outillage loué. Adaptez votre modèle** et nommez la technique employée.

Synthèse

Prendre en compte le temps

Tout se joue sur une seule question : **de quoi la date dépend-elle ?**

Situation	Où va la date	Exemple
La date ne dépend que d'**un objet**	Attribut ordinaire de la relation *Historique : sans objet — il n'y a qu'une date*	`COMMANDE (NumCommande, DateCommande, #NumClient)`
La date dépend d'un **couple d'objets**, l'événement ne survient **qu'une fois**	Attribut de la table de liaison *Historique conservé, par construction*	`OBTENIRDIPLOME (#NumEtu, #NumDip, DateObtention)` — clé : le couple
La date dépend d'un **couple d'objets**, l'événement peut survenir **plusieurs fois**	**Dans la clé primaire** *Historique conservé si et seulement si la date est dans la clé*	`VISITERCLIENT (#NumCli, #NumRep, DateVisite, DureeVisite)` — clé : le triplet

> La phrase à retenir
>
> **Une date hors de la clé primaire s'écrase ; une date dans la clé primaire s'empile.** Tout le reste en découle.

Gérer les attributs non renseignés

- Cette opération met en œuvre l'**héritage** :
  - à partir d'une **table générique** contenant les données communes à tous les objets, on crée des **tables complémentaires** conservant les données propres à chaque catégorie ;
  - les tables complémentaires **héritent de la clé primaire** de la table générique — leur clé primaire *est* leur clé étrangère.
- Le bénéfice ne se limite pas à l'esthétique du modèle : il supprime l'ambiguïté de la case vide et rend possibles des contrôles de saisie par catégorie.

Déterminer la clé primaire dans des situations particulières

Situation	Reconnaissance	Clé primaire
Identification relative	Une numérotation qui **repart à 1** dans chaque objet parent	Concaténation du numéro et de la clé du parent : `EXEMPLAIRELIVRE (NumExp, #CodeLiv, …)` — clé : (`NumExp`, `#CodeLiv`)
Trois objets en relation, sans contrainte	Un employé peut tenir **plusieurs** fonctions sur un projet	Le **triplet** des trois clés étrangères
Trois objets, avec dépendance du couple vers le troisième	Un employé n'a **qu'une** fonction par projet	Le **couple** source ; le troisième attribut devient ordinaire

> ⚠ **Ce que le correcteur regarde en premier**
>
> La clé primaire. Un modèle dont les attributs sont justes mais la clé fausse ne décrit pas la bonne réalité de gestion — et il est sanctionné comme tel. **Soulignez-la, et vérifiez qu'elle garantit l'unicité dans tous les cas de figure**, pas seulement dans l'exemple de l'énoncé.

◀ 2.1 L'organisation d'une base de données · 2.3 Les requêtes SQL ▶